Before we turn our attention to *using* metadata, let's talk about one more way to set custom metadata.

Derived Metadata

As you've worked through the examples in this book, you've always set metadata on one example or group at a time. Sometimes, though, you want to set metadata on *many* examples at once.

For instance, you can mark your quickest-running examples as :fast, and then run just those specs using RSpec's --tag option:

```
$ rspec --tag fast
```

This command would give you a quick overview of your code's health. The set of fast specs would consist of certain hand-picked integration specs, plus everything in spec/unit (since your unit specs are meant to run quickly).

It'd be nice to not have to manually tag each example group in spec/unit with :fast. Fortunately, RSpec supports setting metadata on many examples or groups at once, via its configuration API.

If you add the following code to spec/spec_helper.rb:

08-metadata/05/spec/spec_helper.rb

```
RSpec.configure do |config|
  config.define_derived_metadata(file_path: /spec\/unit/) do |meta|
    meta[:fast] = true
  end
end
```

…RSpec will add the :fast metadata to every example in the spec/unit folder. Let's break down how this code works.

RSpec's define_derived_metdata method checks every example against the *filter expression* we give it. Here, the filter expression is file_path: /spec\/unit/, which means, "match examples defined inside the spec/unit directory."

When the filter expression matches, RSpec calls the passed block. Inside the block, we can modify the metadata hash however we like. Here, we're adding fast: true to the metadata of every matching example. In effect, we're filtering by one piece of metadata (:file_path) in order to set another one (:fast).

In this case, you used a regular expression to find all filenames containing spec/unit. At other times, you may need the values to match exactly. In the following snippet, we want to match all the specs tagged with type: :model to indicate they are testing our Rails models: